

The Gyarados Field Manual

An Electron desktop application that discovers, connects to, and drives LeadFluid peristaltic dispensing pumps over Modbus — across Wi-Fi/TCP and RS-485/RTU serial — for Microlit's hardware line. This document is the complete technical handoff: objectives, architecture, register-level protocol detail, and the state of open issues.

Stack **Electron** · **React 18** · **TypeScript** · **Vite** · **Zustand**

Protocol **Modbus** **TCP** + **RTU**

Repo **gyarados**

Owner **ekansh**

Snapshot **2026-08-27**, commit **92a6f54**

CONTENTS

- | | |
|---------------------------------|------------------------------|
| 01 What This Is | 08 Modbus Register Map |
| 02 Technology Stack | 09 State & the Client Ticker |
| 03 Process Architecture | 10 Known Issues & Fixes |
| 04 Directory Map | 11 Open Questions & Risks |
| 05 Connectivity Engines | 12 Dev & Test Tooling |
| 06 Discovery & Connection Flows | 13 Glossary |
| 07 IPC Contract | |

01 What This Is

Microlit's peristaltic pumps are industrial fluid-dispensing controllers that speak Modbus. Gyarados is the operator console: it finds pumps on the network or on a serial bus, opens a control connection, and gives an operator a dashboard to configure and run dispense cycles.

The application supports two independent transport paths to the same physical pump family, because Microlit ships pumps with either an onboard Wi-Fi/Ethernet Modbus-TCP interface or a wired RS-485 Modbus-RTU interface, and sometimes both are present on a shop floor at once:

- **Wi-Fi / Modbus TCP** — the pump runs a Modbus TCP server on port `502` (or `5020` in development) and is located via a custom UDP broadcast handshake, since it never announces itself unprompted.
- **RS-485 / Modbus RTU** — the pump is one of up to 20 addressable devices on a shared multi-drop serial bus; the app opens the serial port and sweeps unit IDs to find every pump attached.

Once connected, the app polls the pump's registers to drive a live dashboard, and writes control registers (speed, direction, run/pause timing, cycle count, start/stop) back to the hardware. Three pump product tiers exist in the LeadFluid register spec — **base (S-series)**, **clever (L-series)**, and **intelligent (F-series)** — with different register maps and capabilities; today only the base tier has a working dashboard (see [§11](#)).

Primary objective

Give an operator a single console to discover, connect to, and run any Microlit pump regardless of which physical transport it exposes.

Design stance

All networking, serial I/O, and Modbus traffic live in the Electron main process; the renderer is a thin, IPC-driven view layer with no direct hardware access.

Current phase

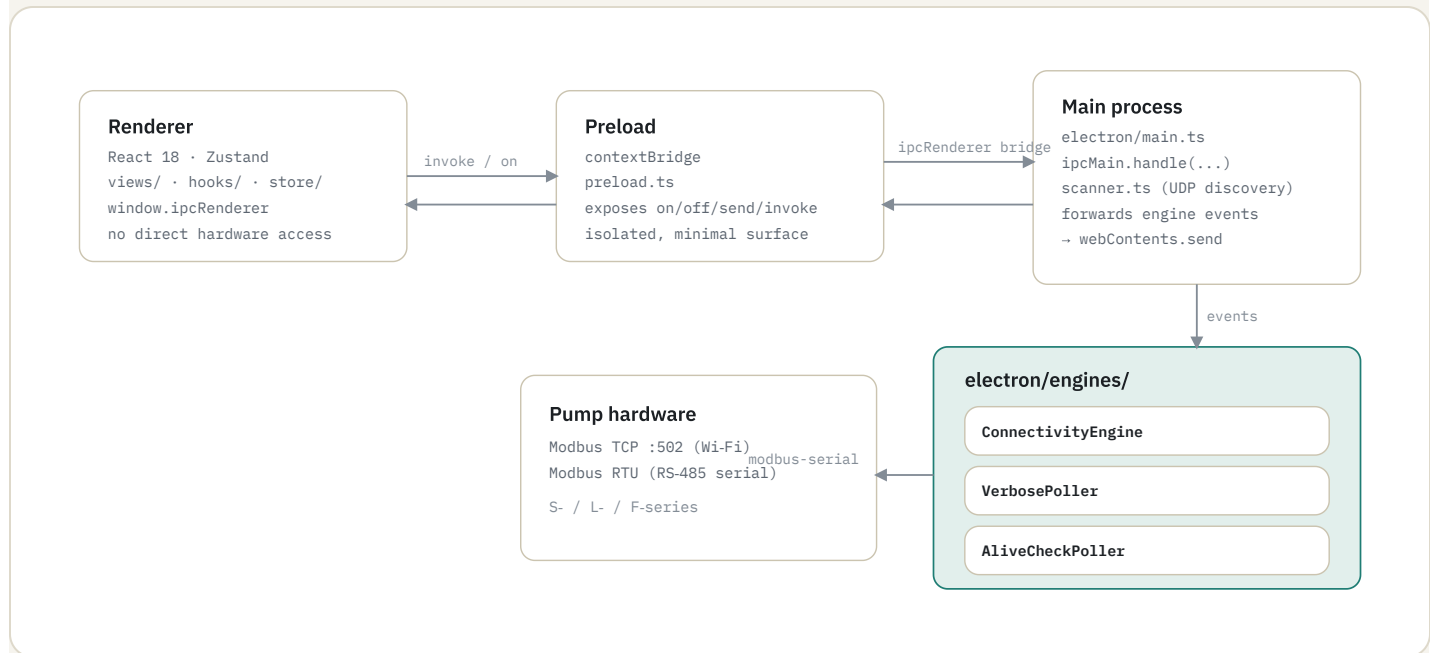
Post-refactor from a monolithic `pumpManager.ts` into modular `electron/engines/` plus a Zustand store — architecture is stable, register coverage is not yet complete.

02 Technology Stack

LAYER	CHOICE	WHY IT'S HERE
Shell	Electron 30	Desktop app with real Node access for raw sockets and serial ports — a browser sandbox can't do either.
UI	React 18 + TypeScript	Renderer process; strict-ish typing across views, hooks, and IPC payloads.
Build	Vite 5 + vite-plugin-electron	Single dev server drives main, preload, and renderer with HMR.
State	Zustand 5	Global store for live pump progress (<code>pumpStore.ts</code>); everything else is local component/hook state.
Styling	Tailwind CSS + shadcn/ui primitives	Utility classes plus a small owned component library in <code>src/components/ui</code> .
Motion	Framer Motion	View transitions, radar scan animation, splash screen.
Modbus (Node)	modbus-serial	One client library for both TCP and RTU-buffered transports.
Serial	serialport	Enumerates and opens OS serial devices for RS-485.
Mock hardware	ESP8266 + ModbusIP_ESP8266	Physical Wi-Fi mock pump for UI development without real hardware (<code>Blink/</code>).

03 Process Architecture

Standard secure Electron three-process split. All Modbus, serial, and network code is Node-only and lives in the main process; the renderer never touches a socket directly.



Boundary responsibilities

- **Renderer** (`src/`) — owns UI state, form inputs, and the local progress-simulation ticker. Talks to the main process only through `window.ipcRenderer.invoke(...)` and `.on(...)`, exposed by preload.
- **Preload** (`electron/preload.ts`) — a deliberately generic `contextBridge` wrapper around `ipcRenderer` (`on/off/send/invoke/removeAllListeners`). It does not allow-list channels; the renderer can call any IPC name, so channel discipline is enforced by convention, not the bridge.
- **Main** (`electron/main.ts`) — registers every `ipcMain.handle` endpoint, owns the `BrowserWindow`, wires engine events to `webContents.send`, and is the only place `SerialPort` and `ModbusRTU` clients are constructed.

04 Directory Map

PATH	CONTENTS
electron/main.ts	App lifecycle, window creation, all <code>ipcMain</code> handlers, engine event → renderer event wiring.
electron/scanner.ts	UDP broadcast discovery for Wi-Fi pumps (adapter-aware).
electron/engines/	<code>ConnectivityEngine</code> , <code>VerbosePoller</code> , <code>AliveCheckPoller</code> , shared <code>types.ts</code> — see §05 .
electron/ipcTypes.ts	Maps internal engine types (<code>PumpStatus</code> , raw device shape) to the UI-facing shape sent over IPC.
electron/preload.ts	<code>contextBridge</code> exposure of a generic <code>ipcRenderer</code> .
src/constants.ts	Every network port, timeout, retry count, and UI delay — single source of truth, imported by both main and renderer.
src/registers.ts	The Modbus register map: <code>MODBUS_REGISTERS</code> , <code>S_SERIES_HOLDING</code> , <code>F_SERIES_INPUT</code> , <code>PUMP_POLL_CONFIG</code> .
src/store/pumpStore.ts	Zustand store + the 100 ms background ticker that simulates live cycle/phase progress for base-series pumps.
src/hooks/useDeviceManager.ts	The renderer's single point of contact with IPC — owns the device list, wires every inbound event, exposes <code>scan/connect/write</code> actions.
src/utils/pumpStateParser.ts	Turns a raw <code>{address: value}</code> register dump into a typed <code>PumpStateData</code> , branching on pump series.
src/views/	<code>ConnectionOptions</code> , <code>DiscoveryView</code> (Wi-Fi), <code>RS485View</code> (serial), <code>ConnectedDevicesView</code> — the pre-dashboard flow.

PATH	CONTENTS
<code>src/components/PumpDashboard.tsx</code>	Dashboard shell: starts/stops verbose polling for the selected pump, dispatches to a series-specific dashboard.
<code>src/components/dashboards/BasePumpDashboard.tsx</code>	The only implemented series dashboard today — timing, RPM, direction, start/stop, live cycle readout.
<code>mockPump.ts</code>	Node/ <code>modbus-serial</code> mock Modbus TCP + UDP server for UI development without hardware.
<code>Blink/</code>	ESP8266 firmware (<code>ModbusIP_ESP8266</code>) — a second, physical mock pump for over-the-air/Wi-Fi testing.
<code>loadTester.ts</code>	Repeats UDP discovery 1,000× to catch intermittent scan failures; logs to <code>loadtest_failures.log</code> .
<code>LEAD FLUID MODBUS QBLF-RDB-2.{pdf,txt}</code>	The manufacturer's Modbus spec for the S-series — the authoritative source for <code>S_SERIES_HOLDING</code> .

05 Connectivity Engines

The 2026-07-31 refactor replaced a single monolithic `pumpManager.ts` with three focused, event-driven singletons. Each is an `EventEmitter` instantiated once and exported directly — there's no dependency injection; modules import each other's singleton.

ConnectivityEngine

The source of truth for what's connected. Owns a single `Map<string, DeviceState>` keyed by `ip` for TCP devices or `"path:unitId"` for RTU devices, plus a per-serial-path client cache so multiple RTU pumps on one bus share a single open `ModbusRTU` connection.

- `connectTCP(ip, port)` — opens the socket, then *validates* the connection by reading the discovery register before declaring the pump `CONNECTED`; a bare TCP handshake is not treated as sufficient (see [§06](#)).
- `scanAndConnectRTU(path, options)` — opens (or reuses) a serial port, then probes unit IDs `1...MAX_RS485_DEVICES` for a response on the discovery register, with `RS485_SCAN_ATTEMPTS_PER_UNIT` retries per ID. A same-settings rescan reuses the already-open client rather than closing and reopening it, and never disconnects devices already known on that path — both were previously sources of flakiness ([§10](#)).
- `writeRegister(id, register, value)` — routes to `client.writeRegisters` directly for TCP, or through the serial lock for RTU.
- `runWithSerialLock(path, fn)` — a mutex per serial path plus an enforced 50 ms minimum gap between actions on that bus, since RS-485 is half-duplex and multi-drop: two engines (or two devices) issuing requests back-to-back on the same wire will collide.

VerbosePoller

High-detail polling for whichever single pump is open in the dashboard right now. Started/stopped explicitly by `PumpDashboard.tsx` on mount/unmount via IPC, at `HEARTBEAT_INTERVAL_MS` (1000 ms). Reads the register blocks defined for the device's `pumpType` in `PUMP_POLL_CONFIG`, inserting a 50 ms gap between blocks on RTU to avoid saturating the bus. Three consecutive failures (`MAX_POLL_FAILURES`) triggers an automatic disconnect.

AliveCheckPoller

Cheap liveness only — a single-register read, never the full poll block — so every connected pump stays "alive" even when its dashboard isn't open. TCP devices are polled individually every `HEARTBEAT_INTERVAL_MS` ; RTU devices are polled per *serial port* every `RS485_CHECK_ALIVE_INTERVAL` (500 ms), sweeping every connected unit on that bus in one pass. Either poller backs off a device that `VerbosePoller` is already actively polling, so a pump never receives two competing streams of Modbus requests.

Engine event catalogue			
EVENT	EMITTED BY	PAYLOAD	CONSUMED BY
status-changed	ConnectivityEngine	{ ip, status }	main.ts → renderer pump-status-changed
device-connected	ConnectivityEngine	DeviceState	AliveCheckPoller (starts polling)
device-disconnected	ConnectivityEngine	id	AliveCheckPoller, VerbosePoller (stop polling)
pump-test-state	AliveCheckPoller	{ ip, state }	main.ts → renderer pump-test-state
pump-fail-count	Verbose/AliveCheckPoller	{ ip, failCount }	renderer fail-indicator UI
pump-state-update	VerbosePoller	{ ip, registers }	main.ts → renderer pump-state-update

06 Discovery & Connection Flows

Wi-Fi / TCP — UDP broadcast discovery

The pump is a pure Modbus *server*; it never advertises itself. Discovery is a bespoke UDP request/reply protocol layered next to Modbus:

1. App broadcasts the ASCII string `DISCOVER_PUMP_REQUEST` on UDP port `5555` — to `255.255.255.255` and to every active IPv4 adapter's own directed broadcast address, computed from that adapter's address and netmask. This adapter-aware step matters on Wi-Fi networks that filter the limited broadcast.
2. Because UDP is fire-and-forget, the app repeats the broadcast `WIFI_DISCOVERY_RETRIES` (3) times, `WIFI_DISCOVERY_RETRY_DELAY_MS` (1000 ms) apart, inside a `WIFI_SCAN_TIMEOUT_MS` (1500 ms) overall window.
3. Pump firmware listens on UDP `5555` and replies `PUMP_ID:<DeviceID>` directly to the sender.
4. The app de-duplicates by both device ID and source IP (a pump can answer from more than one adapter on routed networks) and surfaces each new device to the UI immediately via a callback, not just at the end of the scan.
5. On **Connect**, the app opens a Modbus TCP socket to port `502` (`5020` in dev) with an `800 ms` connect timeout, then reads the discovery register (address `3102`) — only a successful read flips the device to `CONNECTED`. A raw TCP handshake alone is not sufficient; some network paths accept a TCP connection without a live Modbus server behind it.

RS-485 / RTU — unit-ID sweep

1. Operator picks a serial port (filtered from `SerialPort.list()` to exclude bare virtual ports like Linux's `/dev/ttyS0-31`) and sets baud rate, parity, stop bits, and 8 data bits.
2. App opens (or reuses) a single `ModbusRTU` client on that path, then walks unit IDs `1` through `MAX_RS485_DEVICES` (20), reading the discovery register once per ID with up to `RS485_SCAN_ATTEMPTS_PER_UNIT` (2) attempts before giving up on that ID for the pass — RS-485 noise on a multi-drop bus makes a single-shot probe unreliable.
3. All connection events fire only *after* the full sweep completes, so `AliveCheckPoller` can't interleave and interrupt an in-progress scan.
4. A rescan with unchanged port settings reuses the open client and only probes unit IDs not already known — it does not disturb pumps already streaming to an open

dashboard on that bus.

DISCOVERY REGISTER IS DUAL-PURPOSE, BY NECESSITY

Both flows key off the same address, `RS485_DISCOVERY_REGISTER` = `3102` — which is also `S_SERIES_HOLDING.RUN_STATE`. There's no dedicated "are you there" register in the S-series spec, so the app treats "any valid Modbus response" as proof of life. This is deliberate, but it has a real limitation — see [§11](#).

07 IPC Contract

Everything crosses the process boundary through the generic `ipcRenderer` bridge.
These are the channels currently in use.

Renderer → Main (invoke/handle)		
CHANNEL	ARGS	RETURNS
scan-wifi-devices	port?	{ success, devices[] error }
connect-pump	ip, port?	{ success error }
get-serial-ports	—	{ success, ports[] error }
scan-rtu-devices	path, {baudRate,dataBits,parity,stopBits}	{ success, devices[] error }
disconnect-pump	ip	{ success }
write-register	ip, register, value	{ success error }
start-verbose-polling	ip	{ success error }
stop-verbose-polling	ip	{ success error }

Main → Renderer (push events, via webContents.send)		
CHANNEL	PAYLOAD	HANDLED IN
device-found	UiDiscoveredDevice	useDeviceManager
pump-status-changed	{ ip, status: UiPumpStatus }	useDeviceManager
pump-test-state	{ ip, state }	useDeviceManager
pump-state-update	{ ip, registers }	useDeviceManager → pumpStatePars
pump-fail-count	{ ip, failCount }	useDeviceManager
pump-message	string	useDeviceManager → toast

The preload bridge does not allow-list channel names — `on` / `send` / `invoke` all accept any string. Channel discipline is a convention enforced by this table, not by the contextBridge surface itself.

08 Modbus Register Map

Three product tiers share a wire protocol but not a register map. Everything below is what's actually encoded in `src/registers.ts` today, cross-checked against the manufacturer spec where one exists in the repo.

Base / S-series — holding registers (FC 0x03 read, 0x06/0x10 write) — S_SERIES_HOLDING			
NAME	ADDRESS	UNITS	NOTES
RPM	3100	0.1 RPM	e.g. write <code>1500</code> for 150.0 RPM
DIRECTION	3101	enum	0 = CW, 1 = CCW
RUN_STATE	3102	enum	0 = Stop, 1 = Start. Also reused as the universal discovery/liveness probe.
RUN_TIME	3109	0.1 s	Dispense-phase duration
PAUSE_TIME	3110	0.1 s	Pause-phase duration
CYCLES	3111	count	Write-only <i>target</i> per LeadFluid spec §4.3 — confirmed no live decrement/counter behavior. Not a progress readout.

Clever (L-series) / Intelligent (F-series) — input registers (FC 0x04) — F_SERIES_INPUT, sourced from an external manufacturer PDF			
NAME	ADDRESS	WIDTH	MEANING
CURRENT_STEPS	1004	32-bit	Elapsed motor steps
REQ_STEPS	1006	32-bit	Requested/target steps
CURRENT_TIME	1008	—	Not yet mapped into <code>pumpStateParser</code>
CURRENT_VOLUME	1010	32-bit	Not yet mapped into <code>pumpStateParser</code>
CURRENT_CYCLES	1012	32-bit	Live completed-cycle count — unlike the base series, L/F pumps <i>do</i> expose real progress in hardware.

Identity registers — input registers (FC 0x04), identical address across all three series

NAME	ADDRESS	FORMAT	EXAMPLE
Manufacturer string	1018	ASCII, 5 regs / 10 bytes	"Lead Fluid"
Product string	1023	ASCII, 5 regs / 10 bytes	"BT100S" / "BT100L" / "BT100F"

The trailing letter of the product string is the natural series discriminator — S → base, L → clever, F → intelligent — and lines up with the pre-existing "Clever (L-series) / Intelligent (F-series)" comment in `registers.ts`. **Nothing in the codebase reads this register today** (see [§11](#)).

Per-series run-state address — the one register that is *not* at a fixed address

SERIES	RUN-STATE REGISTER
S (base)	3102
L (clever)	4025
F (intelligent)	4126

Poll blocks — PUMP_POLL_CONFIG

What `VerbosePoller` actually reads each cycle, per `pumpType`:

base:	holding [3100-3102], [3109-3111]	input: <i>none</i>
clever:	holding [3100-3102], [3109-3111]	input: [1000-1013]
intelligent:	holding [3100-3102], [3109-3111]	input: [1000-1013]

RTU reads insert a 50 ms gap between blocks (`VerbosePoller.pollRegisters`); TCP does not need the gap since it isn't sharing a half-duplex bus.

09 State & the Client-Side Ticker

The base series has no live progress register in hardware — `CYCLES` (3111) is a write-only target, and the S-series input-register range has no cycle/phase readout at all (confirmed against the LeadFluid spec, §4.1/§4.3). `pumpStore.ts` exists to fill that gap.

How the ticker works

- A single background `setInterval` ticks every 100 ms (one decisecond) for every pump whose local state has `isPlaying: true`.
- It advances `elapsedPhaseTime`, flips `currentPhase` between `dispense` / `pause` against the pump's own `runTimeTarget` / `pauseTimeTarget`, and increments `currentCycle` on each dispense → pause (or dispense → dispense, if no pause phase) transition.
- `syncPumpData(ip, isPlaying, totalCycles, runTime, pauseTime)` is called from `useDeviceManager` on every `pump-state-update` for base pumps — it reconciles the ticker's targets against whatever the hardware currently reports (in case another client changed them), and clamps `currentCycle` so it can never exceed `totalCycles`, the pump's own configured target read back from hardware.
- `clearPumpState(ip)` is called on disconnect (from `useDeviceManager`'s `pump-status-changed` handler) so a disconnected pump's simulated state stops advancing and doesn't leave a stale count behind for the next reconnect.

THIS IS A SIMULATION, NOT A MIRROR

Because there is no hardware signal to reconcile against mid-run, the base-series cycle/phase display is fundamentally a client-side prediction clamped to plausible bounds — not a live readout. If the pump's actual firmware timing drifts from the values it was configured with, or if two clients start the same pump within a poll window of each other, the display can be transiently wrong. This is a hardware limitation of the S-series, not a code defect.

GAP – CLEVER/INTELLIGENT PROGRESS ISN'T SYNCED TO THE STORE

`useDeviceManager.handlePumpStateUpdate` only calls `syncPumpData` when `pumpType === 'base'`. For clever/intelligent pumps, `pumpStateParser` already derives `currentCycle` and `elapsedTime` from the real `CURRENT_CYCLES / CURRENT_STEPS` registers into `device.state`, but nothing forwards it into the Zustand store the way base pumps do. In practice this is currently moot — no dashboard renders for those series yet ([§11](#)) — but it's the next wiring gap once one does; unlike base pumps, L/F progress should be read from hardware directly rather than simulated.

10 Known Issues & Fixes Applied

Two symptom reports — flaky RS-485 discovery, and displayed cycle counts drifting from reality — were traced to root cause on 2026-07-31. Fixes for the addressable parts have shipped; two limitations are inherent to the hardware and are documented rather than "fixed."

RS-485 discovery inconsistency

ROOT CAUSES

Close/reopen race. Rescanning a port used to disconnect every device on it and fire-and-forget close the shared `ModbusRTU` client, then immediately open a new client on the same OS path with no wait — an intermittent race, since closing a serial port is asynchronous at the OS level.

No retry on transient error. The unit-ID sweep gave each ID exactly one probe; any transient error — routine on a noisy multi-drop bus — silently dropped that unit for the whole pass, so repeated scans of an unchanged bus could find different device sets.

Rescan collateral damage. Rescanning to find one new pump tore down every already-connected, already-streaming pump on that same bus first.

FIXES SHIPPED

`scanAndConnectRTU` now reuses the already-open serial client on a same-settings rescan instead of racing a close/reopen, and only tears down + *awaits* a real close when settings actually changed. Rescans no longer disconnect devices already known on the path. Added `RS485_SCAN_ATTEMPTS_PER_UNIT` = 2 retries per unit ID.

Cycle-data desync

ROOT CAUSES

Dead parameter. `syncPumpData` received the pump's real hardware cycle-target readback but discarded it (an underscore-prefixed, unused parameter). Displayed progress was 100% a local ticker with zero reconciliation against the machine.

No cleanup on disconnect. The Zustand `pumpStates` map was never cleared when a pump disconnected (confirmed via a repo-wide grep — no delete/reset call existed anywhere). The 100 ms ticker kept incrementing a disconnected pump's simulated cycle/phase in the background if it was last seen running, so reconnecting to the same pump could show a stale, drifted count.

FIXES SHIPPED

The discarded parameter is now `totalCycles` and is used to clamp the ticker — both at sync time and every 100 ms tick — so displayed `currentCycle` can never exceed the pump's own configured target. Added `clearPumpState(ip)`, wired into `useDeviceManager`'s disconnect handler.

RESIDUAL, HARDWARE-IMPOSED LIMITATION

Reconnecting mid-run to a base pump still cannot recover its true in-progress cycle count — no such live register exists on that series. The clamp bounds the drift; it does not eliminate the gap. This is not fixable in software without a firmware change.

11 Open Questions & Risks

PUMP-TYPE DETECTION IS A STUB

`ConnectivityEngine.connectTCP` and `scanAndConnectRTU` both hard-set `pumpType: 'base'` for every device, explicitly marked `// Default to base for now`. This is by design for now — the owner's call was to defer real detection ("we will identify the pump via discovery afterwards not now") — but it cascades: `VerbosePoller` always loads the `base` poll config, so L/F progress registers are never read for a real clever/intelligent pump, and `PumpDashboard` always renders `BasePumpDashboard` or the "not yet implemented" placeholder.

The register that would fix this is already identified — the product-ID input register at `1023` (§08) — but it's unused anywhere in the codebase today. When this work resumes, remember the run-state address also becomes per-series (§08); the current universal `3102` probe will misread run-state for L/F pumps once they're real.

F-SERIES/L-SERIES REGISTER COVERAGE IS PARTIAL

`F_SERIES_INPUT` covers steps, requested steps, time, volume, and cycles — sourced from an external manufacturer PDF that is **not checked into this repo**. Extending coverage (or building the clever/intelligent dashboard) requires that document. Ask the project owner for it rather than guessing addresses — this app writes control registers directly to physical hardware, so an incorrect guess is unusually costly.

MOCK HARDWARE DOESN'T SHARE ONE REGISTER MAP

Three different simulated pumps exist for development, and they don't agree with each other or with `src/registers.ts`: `mockPump.ts` replies to discovery as `PUMP_ID:9000` and serves `MODBUS_REGISTERS` as defined in the real register map; the ESP8266 firmware in `Blink/` replies `PUMP_ID:8000` and defines its own `PumpRegister` enum (`REG_DEVICE_ID = 0`, `REG_CYCLES_PENDING = 123`) that does not match either. Anyone extending the mocks should treat `src/registers.ts` as the one authoritative map and bring the mocks in line with it, not the other way around.

PRELOAD BRIDGE HAS NO CHANNEL ALLOW-LIST

`contextBridge` exposes generic `on/off/send/invoke/removeAllListeners` wrappers rather than a fixed set of typed methods. Fine for a trusted, single-purpose LAN/serial control app; worth revisiting if this surface ever needs to be hardened (e.g. if the renderer ever loads remote or third-party content).

12 Dev & Test Tooling

SCRIPT / TOOL	COMMAND	PURPOSE
Dev server	<code>npm run dev</code>	Vite + Electron with HMR; Modbus TCP defaults to port <code>5020</code> in this mode (<code>getDefaultModbusPort()</code> checks <code>VITE_DEV_SERVER_URL</code>).
Build	<code>npm run build</code>	<code>tsc && vite build && electron-builder</code> — type-checks before packaging.
Lint/format	<code>npm run lint / format</code>	ESLint (zero warnings allowed) and Prettier over <code>src/</code> and <code>electron/</code> .
Node mock pump	<code>tsx mockPump.ts</code>	Modbus TCP + UDP discovery server on <code>0.0.0.0:5020</code> ; fastest way to develop UI without hardware.
ESP8266 mock pump	<code>flash Blink/Blink.ino</code>	A physical Wi-Fi mock pump — joins SSID <code>MicroLit</code> , serves Modbus TCP on <code>:5020</code> , replies to UDP discovery as <code>PUMP_ID:8000</code> . Useful for exercising the real network/UDP path end-to-end.
Discovery load test	<code>tsx loadTester.ts</code>	Runs <code>scanWifiDevices()</code> 1,000× expecting exactly 2 devices each time; logs every deviation to <code>loadtest_failures.log</code> . Built specifically to catch intermittent UDP scan failures like the ones in §10.

13 Glossary

Modbus TCP	Modbus protocol carried over a TCP socket, typically port 502. Client/server, request-response, no addressing beyond the IP connection itself.
Modbus RTU	Modbus protocol carried over a serial line (here, RS-485), with each device addressed by a numeric unit ID (1–247) on the shared bus.
RS-485	A differential, multi-drop serial standard — many devices share one physical bus, half-duplex, addressed by unit ID. Requires request/response turnaround discipline (see the 50 ms serial lock in §05).
Holding register	Modbus register class that is both readable and writable (FC 0x03/0x06/0x10) — used here for pump control parameters.
Input register	Modbus register class that is read-only from the master's side (FC 0x04) — used here for identity strings and live progress on L/F series.
Unit ID	The RTU address of one device on a shared serial bus; TCP devices are instead addressed by IP and don't need one (the app defaults to unit 1 for TCP).
Base / S-series	Simplest pump tier. No live progress register — the app simulates cycle/phase progress client-side (§09).
Clever / L-series	Mid tier. Shares base control registers; has additional input registers for progress. No dashboard implemented yet.
Intelligent / F-series	Top tier. Same input-register family as clever, with the fullest register set. No dashboard implemented yet.

Compiled from `electron/`, `src/`, `Blink/`, and project history as of commit `92a6f54`.

Prepared for internal handoff — verify register addresses against the manufacturer spec before writing to physical hardware.